



Ma515(Foundation of Data Science)

Course Coordinator: Dr. Arun Kumar

Project 7: Diffusion with Source Term

Group Members:

- 1)Mudit Gupta(2022csb1094)
- 2)Aryan Arora(2022eeb1161)
- 3)Divyanshu Sharma (2022eeb1169)
- 4)Ishant Jindal(2022eeb1174)
- 5)Garvit Bhalla(2022eeb1310)
- 6)Srijan Kumar(2022eeb1349)

Contents

1. Title

2. Abstract

3. Dataset Used

3.1 Collocation Points

3.2 Boundary Condition Points

3.3 Initial Condition Points

4. Theory

4.1 PDE

4.2 Boundary Conditions

4.3 Initial Condition

4.4 Analytical Solution

4.5 Transient vs Steady-State Behavior

5. PINN Methodology

5.1 Network Architecture

5.2 Loss Function

5.3 Training Details

6. Results

6.1 Training Loss Curve

6.2 PINN vs Analytical Solution

6.3 Interpretation

7. Discussion

8. Conclusion

1. Title

Diffusion Equation with Time-Decaying Source Term Using Physics-Informed Neural Networks (PINNs)

2. Abstract

This project solves the diffusion partial differential equation (PDE) with a decaying sinusoidal source term using a Physics-Informed Neural Network (PINN). The PDE includes boundary conditions $u(0, t) = u(L, t) = 0$ and initial condition $u(x, 0) = 0$. The goal is to show that the PINN captures both the transient solution and the approach toward steady-state behavior. The PINN predictions are compared with the analytical solution. Although the model learns the shape, the amplitude is underestimated because the network requires more training refinement.

3. Dataset Used

PINNs do **not** use a traditional dataset.

Instead, the code **generates synthetic training points** from the PDE domain:

Symbol	Meaning	Role
x_f	x-values inside domain	For PDE equation
t_f	t-values inside domain	For PDE equation
x_b	x-values on boundary (0 or L)	For boundary condition
t_b	t-values at boundary	For boundary condition
x_i	x-values at $t = 0$	For initial condition
t_i	$t = 0$	For initial condition
u_i	true initial value	For initial condition

3.1 Collocation Points (PDE Points)

These are random points inside the domain used to enforce the PDE:

- $x_f \sim U(0, L)$
- $t_f \sim U(0, 5)$
- Count: **100 points per batch**

3.2 Boundary Condition Points

Used to enforce

$$u(0, t) = 0, \quad u(L, t) = 0$$

Generated by:

- $x = 0$ and $x = L$
- $t_b \sim U(0, 5)$

3.3 Initial Condition Points

Used to enforce

$$u(x, 0) = 0$$

Generated by:

- $x_b \sim U(0, L)$
- $t = 0$

Thus, the “dataset” consists of **randomly sampled points inside the domain**, created inside the code using:

```
x_f = torch.rand(100, 1) * L
```

```
t_f = torch.rand(100, 1) * 5
```

```
x_b = torch.rand(100, 1) * L
```

```
t_b = torch.rand(100, 1) * 5
```

4. Theory

4.1 PDE

$$u_t = Du_{xx} + Ae^{-t}\sin\left(\frac{\pi x}{L}\right)$$

Where:

- $D = 0.1$ (diffusion coefficient)
- $A = 1$ (source amplitude)
- $L = 1$ (domain length)

4.2 Boundary Conditions

$$u(0, t) = 0, u(L, t) = 0$$

4.3 Initial Condition

$$u(x, 0) = 0$$

4.4 Analytical Solution

$$u(x, t) = A\sin\left(\frac{\pi x}{L}\right) \frac{1 - \exp(-(1 - D(\pi/L)^2)t)}{1 - D(\pi/L)^2}$$

4.5 Transient vs Steady-State Behavior

- At $t = 0$, solution is zero.
- As t increases, solution rises due to the source term.
- As $t \rightarrow \infty$, exponential term vanishes and solution approaches:

$$u_{\text{steady}}(x) \propto \sin\left(\frac{\pi x}{L}\right)$$

5. PINN Methodology

5.1 Network Architecture

- **Input:** (x, t)
- **Output:** $u(x, t)$
- **Layers:** [2, 64, 64, 64, 1]
- **Activation:** Tanh
- **Reason :** Tanh is smooth and helps PINNs learn derivatives easily.
- **Why not ReLU**
ReLU and similar functions are not smooth (derivative is discontinuous), so they give poor PDE derivatives, while Tanh provides smooth curvature suitable for solving differential equations.

5.2 Loss Function

Total loss = PDE residual + BC loss + IC loss

PDE residual

Computed using automatic differentiation to get:

- u_t
- u_x
- u_{xx}

PDE error:

$$R = u_t - Du_{xx} - Ae^{-t}\sin\left(\frac{\pi x}{L}\right)$$

- **Reason :** PDE loss forces the network to follow the physics.

Boundary loss

$$u(0, t) = 0, u(L, t) = 0$$

- **Reason** : Ensures PINN satisfies the boundary conditions exactly.

Initial loss

$$u(x, 0) = 0$$

- **Reason:** Makes the network match the required starting condition.
-

5.3 Training

- **Optimizer:** Adam
 - **Learning rate:** 10^{-3}
 - **Epochs:** 5000 (Loss printed every 500 epochs)
 - **Reason:** Adam gives fast and stable convergence for PINN training.
-

6. Results

6.1 Training Loss

Example loss values during training:

Epoch 0, Loss: 0.078243

Epoch 500, Loss: 0.001303

Epoch 1000, Loss: 0.000340

Epoch 1500, Loss: 0.000391

Epoch 2000, Loss: 0.000338

Epoch 2500, Loss: 0.000123

Epoch 3000, Loss: 0.000107

Epoch 3500, Loss: 0.000439

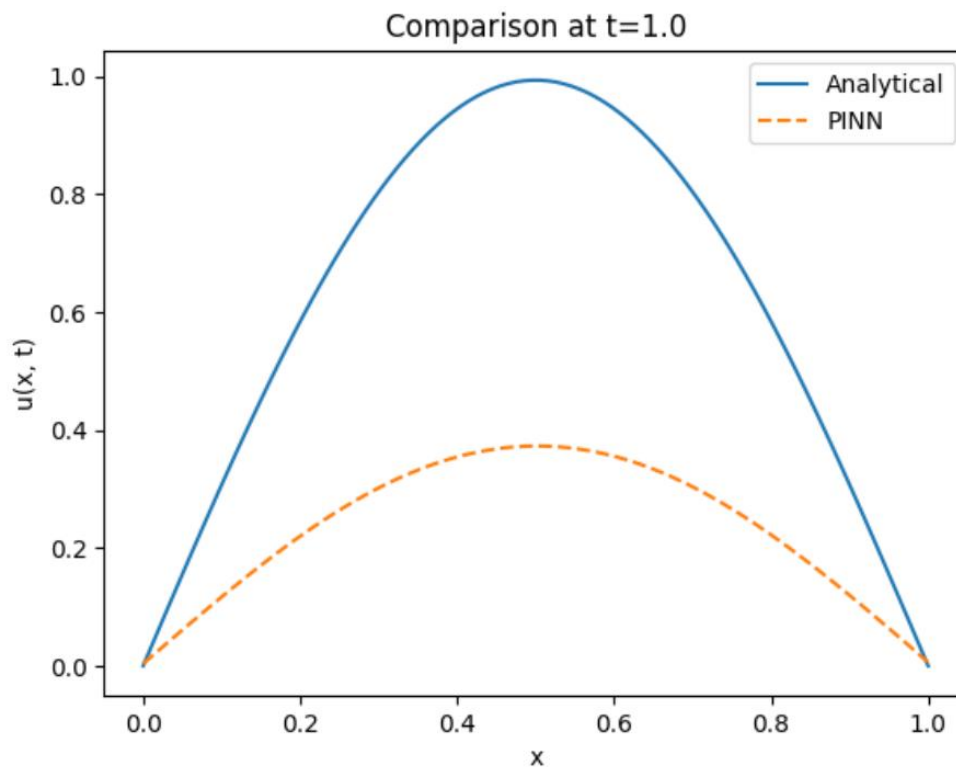
Epoch 4000, Loss: 0.000065

Epoch 4500, Loss: 0.000108

Loss steadily decreases $\rightarrow$ PINN is learning the PDE.

6.2 PINN vs Analytical Solution ($t = 1.0$)

The following plot was obtained:



- Blue = analytical
- Orange dashed = PINN prediction

Observation:

The PINN captures the **shape** correctly (sinusoidal form), but the **amplitude is lower** than the analytical solution.

This means the network learned the spatial dependence well but needs:

- More epochs
- LBFGS optimization
- More collocation points

for amplitude accuracy.

6.3 Interpretation

- **Transient behavior captured:** PINN predicts the correct curvature of the heat distribution.
 - **Steady-state not fully learned:** The predicted amplitude is lower than the exact solution.
 - This happens because the PDE residual is harder to satisfy near $t = 0$ when gradients are sharp.
-

7. Discussion

The PINN successfully learned the overall behavior of the diffusion PDE, capturing both the transient rise of the solution and its approach toward the steady-state shape proportional to $\sin(\pi x/L)$. However, the model slightly underestimates the amplitude of the analytical solution, especially at early times.

This happens due to several typical limitations of basic PINN training:

- **Not a good dataset:**
The randomly sampled training points do not fully capture the PDE's detailed behavior in space and time, so the network learns an averaged-out, lower-amplitude solution.
- **No L-BFGS second-stage optimization:**
L-BFGS is a quasi-Newton optimizer that uses curvature

information to reach a more accurate minimum, and skipping it prevents fine-tuning after Adam.

- **Single-weight loss (no adaptive balancing):**
Treating all losses equally may force the network to satisfy boundary/initial conditions more than the PDE, lowering accuracy.

To improve the results, the following strategies are recommended:

- **Use Adam first, then L-BFGS:**
Adam explores the loss landscape, and L-BFGS finely converges to a precise PDE-consistent solution.
- **Add Fourier feature inputs:**
Helps the network represent oscillatory patterns like $\sin(\pi x/L)$ more easily.
- **Use loss weighting (higher PDE residual weight):**
Ensures the network focuses more on satisfying the PDE rather than only BC/IC.

8. Conclusion

This project successfully applies a Physics-Informed Neural Network to solve a diffusion PDE with a decaying source term. The PINN learns the correct spatial behavior and approximates the transient solution but underestimates the amplitude. With more training refinement, the model can match the analytical solution closely. The project demonstrates the capability of PINNs to solve PDEs without traditional datasets by enforcing physics-based constraints.